Afterimage

Find the document from what you remember,
even when you cannot remember its name.

A human-memory document retrieval system

Project summary, Advanced Data Structures and Information Retrieval

Souryam Bandyopadhyay

C++20, Python 3, JavaScript, SQLite. Runs locally on macOS and Linux.

1 The problem with finding files

Afterimage is a search system for people who remember only a small part of a file. You might recall a single word, a sentence, a topic, a spelling mistake, or roughly when you opened it, but not what it was called. Ordinary file search often fails in exactly this situation.

> **Why "Afterimage"**
> *An afterimage is the faint impression that stays in your eye after you look away. The application works the same way: it uses the small clues that remain in memory to bring a document back.*

Where ordinary search falls short

- **The name is forgotten.** Without the exact file name, a normal search has nothing to match.
- **One typo breaks it.** A small spelling slip can hide the right file completely.
- **Scanned pages have no text.** Images and scanned PDFs need OCR before their words can be searched.
- **Results arrive without context.** A plain list does not say why a file matched or what else is related.

What the project sets out to do

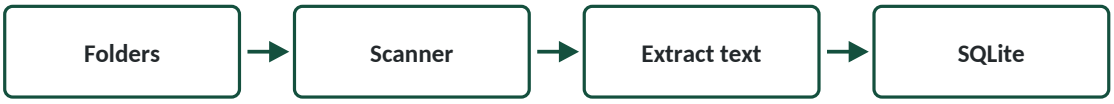
Goal	What Afterimage does
Collect documents	Scans folders and stores the supported files in a library.
Read the text	Pulls text from text files, PDFs and images using extraction tools and OCR.
Search flexibly	Finds exact words, prefixes, middle fragments, small typos and phrases.
Rank sensibly	Orders results and explains each score.
Show relations	Suggests other documents linked by important concepts.
Stay private	Keeps documents and all processing on the user's own computer.

> **Scope**
> *Built for one user on macOS or Linux, with a local folder library and a browser interface on the same machine. Cloud sync, multi-user access and handwriting recognition are outside the current scope.*

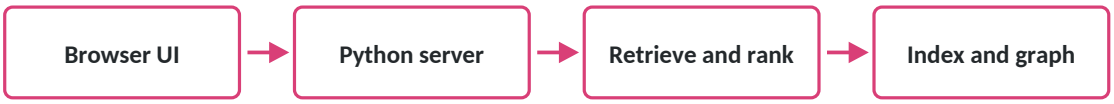
2 How the system works

The C++ engine does the heavy lifting: scanning, extraction, indexing and search. A Python server connects that engine to the browser, and SQLite keeps the library on disk.

Indexing: new and changed files only



Search: every query



Two paths run through the system. Indexing happens once per file; searching happens on every query.

From folder to result

Step	What happens
1. Scan	The application visits folders and picks out supported files: TXT, MD, PDF, PNG and JPG.
2. Extract	Text is read directly from text files and ordinary PDFs. Tesseract OCR reads images and scanned PDF pages.
3. Store	File details and extracted text are saved in SQLite, so the library survives a restart.
4. Index	The engine builds a word-to-document index in memory so searches are fast.
5. Search	Several retrieval methods collect candidates. BM25 and a few extra signals put them in order.
6. Display	The browser shows ranked files, highlighted passages, page details and related documents.

The two data paths

- **Indexing:** folder, scanner, text extraction or OCR, then SQLite. New and changed files are processed; unchanged files are skipped.
- **Searching:** query, in-memory index, candidate retrieval, ranking, concept graph, then results.

> **Why SQLite is the source of truth**
> *The search index and concept graph are temporary and get rebuilt from stored data. That keeps everything consistent and avoids running a separate database server.*

3 Data structures and algorithms

This is the core of the project. It brings together maps, hash tables, vectors, dynamic programming, ranking and graph traversal.

Positional inverted index

An inverted index lists, for every word, the documents it appears in. Afterimage also stores each word's position. The sentence "raft elects a leader" becomes raft(0), elects(1), a(2), leader(3). Those positions let the system check whether words sit next to each other.

Ordered map for prefix search

The vocabulary lives in a std::map, so words stay sorted. lower_bound() jumps to the first candidate and the system reads forward while words still start with the typed prefix. "travers" finds "traversal" and "traversing". The reported cost is O(log V + k), where V is the vocabulary size and k the number of matches.

Infix search

Finds a fragment from the middle of a word: "ntersection" matches "intersections". It checks vocabulary terms and caps the number of matches to keep the work bounded.

Fuzzy search with Levenshtein distance

Levenshtein distance counts the fewest insertions, deletions or substitutions needed to turn one word into another. "schedulng" is one insertion away from "scheduling". Dynamic programming computes it, with a length check, two DP rows and early stopping to avoid wasted work.

Phrase search with word positions

A phrase matches only if its words appear consecutively and in order. If the first word is at position p, the next must be at p+1, then p+2, and so on. Binary search runs over the stored position lists.

BM25 ranking

BM25 ranks the candidates. It favours useful matching words, especially ones that are rare in the library, and accounts for document length so a very long file does not win by default. Small boosts are added for title matches, file-path matches, exact phrases, query coverage and recent modification time.

Concept graph and BFS

A graph links each document to the important words it contains, with common words removed so they do not create meaningless links. Breadth-first search walks these links to find other documents that share useful concepts with the top results.

4 Implementation and interface

Each technology has one job. That keeps the search engine separate from the interface and easier to maintain.

Technology	Role in the project
C++20	The engine: scanning, extraction control, indexing, retrieval, ranking and graph operations.
STL containers	std::map, unordered_map, unordered_set, vector and queue back the core data structures.
SQLite3	Stores document records, extracted text and optional semantic vectors.
Poppler	Reads text from PDFs and turns scanned PDF pages into images for OCR.
Tesseract OCR	Recognises text in images and scanned PDF pages.
Python 3	Runs the local HTTP server, interprets natural-language queries and combines result signals.
HTML, CSS, JavaScript	The browser-based search and library interface.
Ollama (optional)	Creates local text embeddings for meaning-based semantic search.

What the interface offers

- **Memory Search:** search box, filters, ranked result cards, highlighted text, page number, score explanation and related memories.
- **Library:** start indexing a folder, watch live progress and logs, browse stored documents.
- **Developer Console:** document count, database size, terms and postings, plus safe read-only SQL queries.
- **Document reader:** open a result, inspect its extracted pages, and edit its title, tags or personal notes.

Natural-language search

The Python query parser understands phrases such as “last week”, file-type words like PDF or JPG, and common typing mistakes. It drops conversational filler and keeps the terms that matter. The optional semantic layer uses local embeddings to find similar meaning even when the exact words differ.

> **Privacy and safety**
> *The web server listens only on 127.0.0.1. External programs start without passing commands through a shell. File size, output size and processing time are limited, and a problem file never halts a full scan. SQLite uses prepared statements and transactions to protect the stored data.*

5 Testing, results and conclusion

The report records 12 automated CTest groups, and all 12 passed. Together they contain about 290 assertions covering document management, folder scanning, SQLite storage, text and OCR extraction, indexing, retrieval, ranking, graph operations, full-pipeline integration and command-line workflows.

Example queries

Query	What it shows
schedulng	Fuzzy search finds “scheduling” despite a one-letter spelling error.
ntersection	Infix search finds a word from a middle fragment.
raft leader heartbeat	Word matching, prefix expansion and phrase evidence combine; related documents may be suggested.
a java assignmnet that I opened last week	The web layer corrects a common typo, identifies the topic and applies a time filter.

Benchmark figures

Library	Documents	Queries with results	Average query time
Bundled sample and test corpora	25	26 of 50	about 4 ms
Author’s own library	2,185	30 of 50	about 60 ms

Read the benchmark carefully

A query counts as successful if it returns at least one result. That does not prove the correct document ranks first. The report names a labelled evaluation set and stronger ranking measures as the next step.

Limitations

- Semantic search is optional; normal search is mainly based on words and fragments.
- OCR may struggle with handwriting, poor scans, diagrams and complex tables.
- The index is rebuilt in memory at the start of each search session, which may slow down for very large libraries.
- The query parser relies on hand-written rules and will miss some new phrasings.
- The concept graph is a useful heuristic, not proof that two documents are truly related.

Future improvements

Save the index permanently, improve language understanding, add more accurate semantic ranking, support handwriting OCR, extract information from tables and diagrams, add Windows support, and watch folders automatically.

Conclusion

Afterimage shows how core data structures and algorithms solve a practical problem. It combines a positional inverted index, ordered maps, fuzzy matching through dynamic programming, BM25 ranking and a document-concept graph searched with BFS, backed by text extraction, OCR, SQLite storage and a browser interface. Its main contribution is helping people find files from incomplete memories while their documents stay on their own computer.

References

1. Robertson and Zaragoza, “The Probabilistic Relevance Framework: BM25 and Beyond”, 2009.
2. Manning, Raghavan and Schütze, *Introduction to Information Retrieval*, 2008.
3. Levenshtein, “Binary codes capable of correcting deletions, insertions and reversals”, 1966.
4. Cormen et al., *Introduction to Algorithms*.
5. Tesseract OCR, SQLite, Poppler and Ollama documentation.